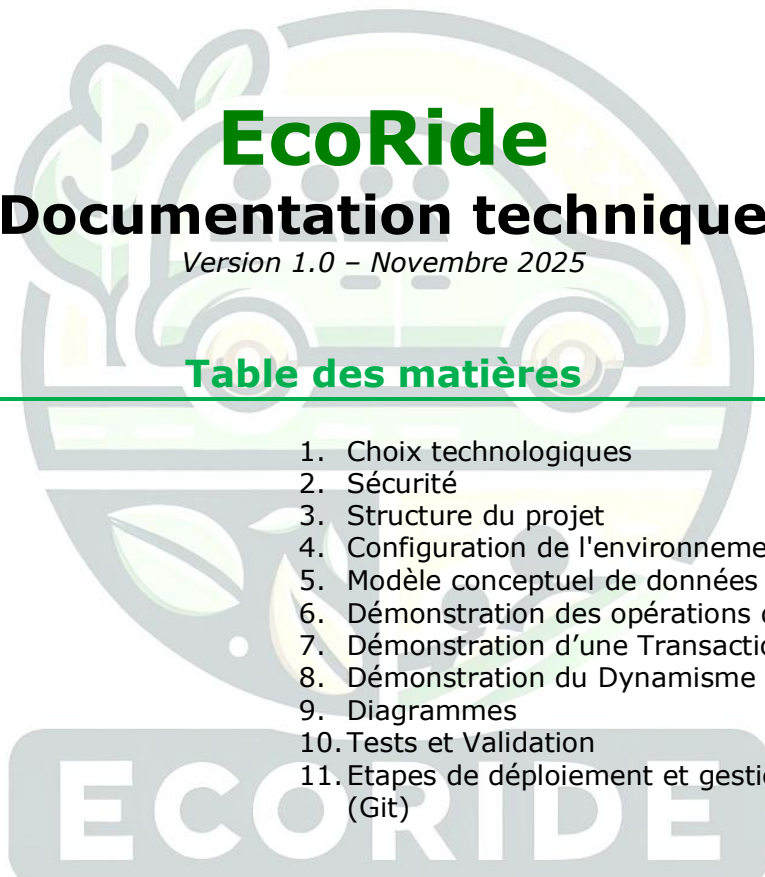




EcoRide

Documentation technique

Version 1.0 – Novembre 2025

Table des matières

1. Choix technologiques
2. Sécurité
3. Structure du projet
4. Configuration de l'environnement
5. Modèle conceptuel de données
6. Démonstration des opérations de CRUD
7. Démonstration d'une Transaction Métier
8. Démonstration du Dynamisme Côté Client
9. Diagrammes
10. Tests et Validation
11. Etapes de déploiement et gestion de version (Git)

1. Choix technologiques l'application

Le développement de l'application EcoRide a été guidé par des choix technologiques visant un équilibre entre la robustesse, la rapidité de développement et la conformité avec les exigences du cahier des charges.

- **Langage Serveur (Backend) : PHP natif**

- **Choix :** L'application a été développée en PHP natif (version 8.1+) en suivant une architecture de type "Contrôleur Frontal" inspirée du modèle MVC.
- **Justification :** Ce choix a été fait pour démontrer une maîtrise des concepts fondamentaux du développement web backend (routage, gestion des requêtes HTTP, séparation de la logique et de l'affichage) sans dépendre d'un framework majeur comme Symfony ou Laravel. L'utilisation de **PDO** (PHP Data Objects) garantit un accès sécurisé et standardisé à la base de données.

- **Interface Utilisateur (Frontend) : HTML, CSS, Bootstrap & JavaScript**

- **Choix :** L'interface a été construite avec une stack classique et éprouvée.
- **Justification :**

Bootstrap 5 a été choisi pour son système de grille puissant et ses composants prêts à l'emploi (comme la barre de navigation, les cartes de trajet et les modales). Cela a permis de développer rapidement une interface responsive et moderne.

Sass (SCSS) a été utilisé pour rendre le code CSS plus maintenable. Grâce aux **variables**, la charte graphique est centralisée ; grâce à **l'imbrication** (nesting), le code est plus lisible ; et grâce aux **partials** (`_header.scss`, `_covoiturage.scss`), les styles sont organisés en fichiers logiques.

Le **JavaScript natif** est utilisé pour les interactions dynamiques (appels API via `fetch`, gestion des modales), gardant l'application légère sans la complexité d'un framework front-end lourd.

- **Bases de Données : Approche Hybride (MySQL & MongoDB)**

- **Choix :** Conformément à une exigence clé de l'énoncé, une architecture de données hybride a été mise en place.
- **Justification :**

MySQL a été choisi pour la partie relationnelle en raison de sa grande fiabilité, de sa robustesse et de ses garanties transactionnelles (ACID), ce qui est idéal pour les données structurées et critiques (utilisateurs, trajets, réservations).

MongoDB a été utilisé pour la partie non-relationnelle afin de gérer les préférences des utilisateurs. Sa flexibilité et sa structure "sans schéma" sont parfaitement adaptées pour stocker une liste de préférences qui peut inclure des valeurs personnalisées ajoutées par l'utilisateur, sans nécessiter de modification de la structure de la base.

- **Gestion des Dépendances : Composer & NPM**

- **Choix** : Composer pour le PHP et NPM pour le JavaScript sont les standards de l'industrie.
- **Justification** : Leur utilisation permet de gérer proprement les librairies externes (comme PHPMailer ou Bootstrap), d'assurer des installations reproductibles grâce aux fichiers de "lock", et de faciliter la maintenance du projet.



2. Sécurité

La sécurité a été un pilier central du développement. L'architecture a été conçue pour prévenir les failles majeures identifiées par le **Top 10 de l'OWASP**.

A. Prévention des Injections (OWASP A03:2021 - Injection)

La faille la plus critique pour une application gérant des données est l'injection SQL.

Problème : Un attaquant pourrait injecter du code SQL malveillant dans un formulaire (par exemple, dans le champ email de la connexion) pour contourner l'authentification ou détruire la base de données.

Solution : Aucune donnée utilisateur n'est jamais directement concaténée dans une requête. L'application utilise **systématiquement les requêtes préparées** via l'objet **PDO**. Les données sont envoyées séparément de la requête, garantissant qu'elles sont toujours traitées comme du texte et jamais comme du code exécutable.

Exemple de code (extrait de `UserModel::findUserByEmail`) :

```
public function findUserByEmail(string $email)
{
    // Extrait de UserModel::findUserByEmail()

    // 1. Préparation : Le '?' est un marqueur de position sécurisé.
    // La requête SQL est envoyée au serveur SANS les données.
    $stmt = $this->pdo->prepare(
        "SELECT id, mot_de_passe, role, actif
FROM utilisateurs
WHERE email = ?"
    );

    // 2. Exécution : Les données (l'email) sont envoyées séparément.
    // La base de données reçoit la donnée et l'insère dans le '?'
    // en la traitant comme une simple chaîne de caractères,
    // neutralisant toute injection.
    $stmt->execute([$email]);
    return $stmt->fetch();
}
```

B. Hachage des Mots de Passe (OWASP A07:2021 - Identification Failures)

- **Problème** : Stocker des mots de passe en clair (ou avec des algorithmes obsolètes comme MD5/SHA1) dans la base de données. En cas de fuite de données, tous les comptes sont compromis.
- **Solution** : L'application utilise les fonctions natives sécurisées de PHP, `password_hash()` et `password_verify()`, qui appliquent l'algorithme **Bcrypt** (l'algorithme par défaut, robuste et lent).

Exemple de code (création et vérification) :

```
<?php

// Extrait de AuthController::register()
// 1. HACHAGE (Inscription)
// On génère un "hash" salé et sécurisé du mot de passe.
$hash = password_hash($mdp, PASSWORD_DEFAULT);

// 2. STOCKAGE
// C'est ce hash (ex: "$2y$10$...") qui est stocké en BDD.
$userModel->createUser($pseudo, $email, $hash, ...);

// Extrait de AuthController::login()
// 1. RÉCUPÉRATION
// On récupère le hash stocké en BDD pour cet email.
$user = $userModel->findUserByEmail($email);

// 2. VÉRIFICATION
// password_verify() compare le mot de passe en clair fourni
// au hash stocké, en utilisant le sel contenu dans le hash.
if ($user && password_verify($password, $user['mot_de_passe'])) {
    // Mot de passe correct
    $_SESSION['user_id'] = $user['id'];
    // ...
}
```

C. Contrôle d'accès (OWASP A01:2021 - Broken Access Control)

Problème : Permettre à un utilisateur d'accéder à des ressources qui ne lui appartiennent pas (par exemple, un passager accédant à la page admin en tapant l'URL /admin).

Solution : Un "Gardien de Sécurité" est implémenté au début de chaque route ou script sensible. Ce gardien vérifie la présence et la valeur de la variable \$_SESSION['user_role'] avant d'autoriser l'exécution du code.

Exemple de code (extrait du routeur index.php) :

```
<?php

// Extrait du routeur (index.php)

// ...

case '/employees':
    // 1. LE GARDIEN
    // On vérifie si l'utilisateur est connecté ET s'il a le bon rôle.
    if (!isset($_SESSION['user_id']) || $_SESSION['user_role'] !== 'employe') {
```

```

        // 2. REDIRECTION
        // Si l'accès n'est pas autorisé, l'utilisateur est
        // renvoyé à la page de connexion sans voir la page.
        header('Location: /login');
        exit();
    }

    // 3. ACCÈS AUTORISÉ
    // Le code n'est exécuté que si le gardien est passé.
    EmployeeController::showDashboard($pdo);
    break;

case '/admin':
    // Le gardien est différent pour le rôle 'admin'
    if (!isset($_SESSION['user_id']) || $_SESSION['user_role'] !== 'admin') {
        header('Location: /login');
        exit();
    }
    renderView('admin');
    break;

```

D. Sécurisation des Identifiants

Problème : Écrire en dur les mots de passe de la base de données ou les clés d'API dans les fichiers PHP (ex: Core/Database.php) et les "commiter" sur Git.

Solution : Utilisation de la bibliothèque Dotenv pour stocker tous les identifiants sensibles dans un fichier .env qui est **exclu du dépôt Git** (via le fichier .gitignore).

Exemple de code (configuration) :

```

<?php

// Extrait de index.php (au démarrage)
if (file_exists(__DIR__ . '/.env')) {
    $dotenv = Dotenv\Dotenv::createImmutable(__DIR__);
    $dotenv->load();
}

// Extrait de Core/Database.php
// Les identifiants sont lus depuis les variables d'environnement
$dsn = "mysql:host=" . $_ENV['DB_HOST'] . ";dbname=" . $_ENV['DB_NAME'];
$this->pdo = new PDO($dsn, $_ENV['DB_USER'], $_ENV['DB_PASS']);

```

Fichier .gitignore (extrait) :

```

# Fichier de variables d'environnement sensibles
.env

```

3. Structure du projet

L'application EcoRide est structurée selon le patron d'architecture "**Front Controller**" (Contrôleur Frontal), avec une organisation des dossiers inspirée du modèle **MVC (Modèle-Vue-Contrôleur)**. Toutes les requêtes HTTP sont interceptées et redirigées par le serveur web vers un point d'entrée unique, le fichier `index.php`, qui agit comme un routeur principal.

L'arborescence du projet est organisée comme suit :

- **/assets/**
Contient tous les fichiers statiques (ressources front-end) comme les feuilles de style CSS compilées et le dossier images.
- **/Node-modules/**
Contient tous les fichiers BOOTSTRAP (ignoré par Git).
- **/src/**
C'est le répertoire principal du code source de l'application en PHP.
 - **/src/Controllers/** : Contient les classes des **Contrôleurs**. Chaque contrôleur regroupe la logique métier pour une section de l'application (ex: UserController, TrajetController). Ils reçoivent les requêtes du routeur, interagissent avec la base de données, et préparent les données pour les vues.
 - **/src/Core/** : Contient les classes fondamentales de l'application, notamment Database.php, qui utilise le design pattern Singleton pour gérer une connexion unique et efficace aux bases de données MySQL et MongoDB.
 - **/src/Models/** : Contient les classes des Modèles (ex: UserModel, TrajetModel). C'est la seule couche de l'application autorisée à communiquer avec la base de données
- **/vendor/**
Ce dossier est géré automatiquement par **Composer**. Il contient toutes les bibliothèques PHP externes téléchargées et n'est pas inclus dans le dépôt Git.
- **/views/**
 - Ce dossier contient toutes les **Vues** : Ce sont des fichiers `.php` dont le rôle est d'afficher le code HTML et les données préparées par les contrôleurs pour construire l'interface utilisateur.
- **header_template.php** : Fichier de template contenant l'en-tête HTML commun à toutes les pages, y compris la barre de navigation principale et le chargement des feuilles de style.
- **footer_template.php** : Fichier de template pour le pied de page commun, qui contient également le code HTML de la modale de réservation et les appels aux scripts JavaScript globaux.
- **index.php** : Le **Contrôleur Frontal**. C'est le cœur de l'application qui reçoit toutes les requêtes, analyse l'URL demandée, et appelle le bon contrôleur pour exécuter la logique appropriée.
- **.htaccess** : Fichier de configuration pour le serveur web Apache. Son rôle est crucial : il redirige toutes les URL non existantes physiquement (comme `/profile` ou `/login`) vers le fichier `index.php`.
- **composer.json** : Fichier de manifeste qui déclare toutes les dépendances PHP du projet (PHPMailer, Dotenv, etc.), gérées par Composer.
- **package.json** : Fichier de manifeste pour les dépendances front-end (Bootstrap, Bootstrap Icons), gérées par npm.
- **.env** : Fichier de configuration local (ignoré par Git) contenant les variables d'environnement sensibles.
- **Readme.md** : Le document principal du projet, présentant l'application et fournissant les instructions détaillées pour l'installation en environnement de développement local.

Cette structure permet une séparation claire des préoccupations (logique, affichage, configuration), ce qui rend le projet plus facile à maintenir et à faire évoluer.

4. Configuration de l'environnement

Pour exécuter le projet en local, suivez ces étapes :

1. Cloner le dépôt :

```
Bash
git clone https://github.com/Fani1987/EcoRide.git
cd EcoRide
```

2. Installer les dépendances :

```
Bash
composer install
npm install
```

3. Configurer les variables d'environnement :

- Créez un fichier `.env` à la racine du projet en copiant le modèle `.env.example` (si vous en avez un) ou en le créant manuellement. Remplissez-le avec vos informations locales :

```
# Base de données MySQL
○ DB_HOST=localhost
○ DB_NAME=ecoride
○ DB_USER=root
○ DB_PASS=
○ DB_CHARSET=utf8mb4

# Base de données MongoDB
○ MONGO_DB_HOST=localhost
○ MONGO_DB_PORT=27017
○ MONGO_DB_NAME=ecoride
○ MONGO_DB_USERNAME=
○ MONGO_DB_PASSWORD=

# Service d'e-mails (exemple avec Mailtrap pour le test)
○ MAIL_HOST=sandbox.smtp.mailtrap.io
○ MAIL_USERNAME=xxxxxxxxxxxxxx
○ MAIL_PASSWORD=xxxxxxxxxxxxxx
○ MAIL_PORT=2525
```

4. Importer la base de données :

- Créez une base de données nommée `ecoride` dans votre gestionnaire de base de données (phpMyAdmin).
- Importez le fichier `ecoride.sql` (fourni à la racine du projet) dans cette base de données.

5. Configurer le serveur web (Très important) :

- Configurez un **Hôte Virtuel** (Virtual Host) dans votre environnement WAMP ou XAMPP qui pointe vers le dossier racine de votre projet.
- Accédez ensuite au site via l'URL que vous avez définie (ex: `http://ecoride.local`). Cela est nécessaire pour que le routage des URL (`/profile`, `/login`, etc.) fonctionne correctement grâce au fichier `.htaccess`.

5. Modèle conceptuel de données

L'architecture des données d'EcoRide repose sur une approche hybride, utilisant à la fois une base de données relationnelle (MySQL) et une base de données non-relationnelle (NoSQL - MongoDB). Cette conception permet de tirer parti des forces de chaque technologie : la structure et la fiabilité de SQL pour les données critiques, et la flexibilité de NoSQL pour les données semi-structurées.

○ Base de Données Relationnelle (MySQL)

Le cœur de l'application est géré par une base de données MySQL. Elle garantit l'intégrité et la cohérence des données transactionnelles grâce à un schéma strict et à l'utilisation de clés étrangères.

Les tables principales sont :

- **utilisateurs** : C'est la table centrale qui stocke les informations de tous les comptes, qu'il s'agisse de passagers, de chauffeurs, d'employés ou d'administrateurs. Elle contient les identifiants de connexion, le rôle, le solde de crédits et la note moyenne pour les chauffeurs.
- **covoiturages** : Contient toutes les offres de trajet publiées par les chauffeurs. La colonne statut (planifié, en_cours, terminé, annulé) est essentielle pour gérer le cycle de vie de chaque trajet.
- **vehicules** : Stocke les informations sur les véhicules des chauffeurs. Un chauffeur peut posséder plusieurs véhicules.
- **reservations** : Table de liaison qui matérialise la participation d'un passager à un covoiturage. Sa propre colonne statut (en_attente, confirmée, validée, annulée, etc.) est cruciale pour gérer le processus de réservation, de la demande initiale à la validation finale.
- **avis** : Contient les notes et commentaires laissés par les passagers. Son champ statut permet de mettre en place le flux de modération par les employés.
- **incidents et notifications** : Tables de support qui permettent respectivement de gérer les litiges et d'envoyer des informations aux utilisateurs sur leur profil. Des **clés étrangères** et des contraintes d'intégrité (comme ON DELETE CASCADE) sont utilisées pour garantir que les relations entre ces tables restent toujours cohérentes.

○ Base de Données Non-Relationnelle (MongoDB)

Pour répondre au besoin de flexibilité des préférences de voyage des utilisateurs (US 8), une base de données NoSQL a été mise en place avec MongoDB.

- **Objectif** : Gérer les préférences des utilisateurs, qui peuvent être à la fois des options standards (Fumeur, Animal, Musique) et des valeurs personnalisées ajoutées par l'utilisateur ("Discussions bienvenues", "Petit bagage", etc.).
- **Structure** : Une collection unique nommée preferences est utilisée. Chaque document de cette collection correspond à un utilisateur et contient deux champs principaux :
 - mysql_user_id : L'ID de l'utilisateur provenant de la table SQL utilisateurs, qui sert de lien entre les deux bases de données.

preferences : Un tableau de chaînes de caractères contenant la liste des préférences de l'utilisateur.

- **Avantage :** Cette approche "sans schéma" permet d'ajouter n'importe quelle préférence personnalisée sans jamais avoir à modifier la structure de la base de données, offrant une flexibilité maximale. En conclusion, cette architecture de données hybride permet à l'application EcoRide d'être à la fois robuste et structurée pour ses données essentielles, tout en étant souple et évolutive pour les données plus dynamiques comme les préférences des utilisateurs.



6. Démonstration des Opérations CRUD

L'acronyme CRUD désigne les quatre opérations fondamentales de la persistance des données. Pour illustrer leur implémentation dans l'architecture MVC d'EcoRide, nous allons suivre le cycle de vie de deux entités clés : le Véhicule (pour C, R, U) et le Trajet (pour D).

1. CREATE : Ajouter un nouveau véhicule

Cette opération est initiée par un chauffeur depuis son profil.

- **Flux** : Le formulaire de profil envoie les données à la route `/api/updateVehicle`. Le `UserController` reçoit ces données. S'il ne détecte **pas** de `vehicule_id`, il comprend qu'il s'agit d'une création et appelle `VehiculeModel::create`.

Extrait de code (Orchestration - `UserController.php`):

```
<?php

// Extrait de UserController::updateVehicle()

public static function updateVehicle(PDO $pdo, int $userId, array $postData)
{
    // ... (Récupération des variables $marque, $modele, etc.)

    $vehiculeId = $postData['id'] ?? null;
    $vehiculeModel = new VehiculeModel($pdo);
    $success = false;

    if ($vehiculeId) {
        // Logique de l'UPDATE (voir section 3)
        // ...
    } else {
        // --- LOGIQUE CREATE ---
        // 1. Le Contrôleur appelle le Modèle
        $success = $vehiculeModel->create(
            $userId,
            $marque,
            $modele,
            $couleur,
            $plaque,
            $energie,
            $date_immat
        );
        $message = 'Véhicule ajouté.';
    }

    // ... (Réponse JSON)
}
```

Extrait de code (Accès aux données - VehiculeModel.php)

```
<?php
```

```
// Extrait de VehiculeModel::create()
```

```
public function create(int $userId, string $marque, string $modele, string
$couleur, string $plaque, string $energie, string $immat): bool
{
    // 2. Préparation : La requête SQL est préparée avec des '?'
    $stmt = $this->pdo->prepare(
        "INSERT INTO vehicules (utilisateur_id, marque, modele, couleur,
plaque_immatriculation, energie, date_premiere_immat)
        VALUES (?, ?, ?, ?, ?, ?, ?)"
    );

    // 3. Exécution : Les données sont envoyées en toute sécurité
    return $stmt->execute([$userId, $marque, $modele, $couleur, $plaque,
$energie, $immat]);
}
```

2. READ : Lire les véhicules d'un utilisateur

Cette opération est initiée lors du chargement de la page de profil.

- **Flux** : Le UserController::showProfilePage (le "chef d'orchestre") détermine si l'utilisateur est chauffeur. Si oui, il appelle VehiculeModel::findByUserId pour récupérer la liste de ses véhicules et la transmet à la vue.

Extrait de code (Orchestration - UserController.php)

```
<?php
```

```
// Extrait de UserController::showProfilePage()
```

```
public static function showProfilePage(PDO $pdo, int $userId)
{
    // ... (Instanciation de $userModel, $avisModel, etc.)
    $vehiculeModel = new VehiculeModel($pdo);

    // ... (Récupération de $user, $preferences, etc.)

    $vehicules = [];
    if ($user['est_chauffeur']) {
        // 1. Le Contrôleur appelle le Modèle pour lire les données
        $vehicules = $vehiculeModel->findByUserId($userId);
    }

    // 2. Le Contrôleur prépare les données pour la vue
    $data = [
```

```

        'user' => $user,
        'vehicules' => $vehicules,
        // ...
    ];

    // 3. Appel de la vue
    \renderView('profile', $data);
}

```

Extrait de code (Accès aux données - VehiculeModel.php)

```

<?php

// Extrait de VehiculeModel::findById()

public function findById(int $userId)
{
    // 1. Préparation de la requête de LECTURE
    $stmt = $this->pdo->prepare("SELECT * FROM vehicules WHERE utilisateur_id
= ?");

    // 2. Exécution sécurisée
    $stmt->execute([$userId]);

    // 3. Récupération des résultats
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

```

3. UPDATE : Modifier un véhicule existant

Cette opération utilise la même route /api/updateVehicle que le "CREATE".

- **Flux** : Le formulaire envoie les données (y compris le vehicle_id caché). Le UserController détecte la présence du vehicle_id et comprend qu'il s'agit d'une modification. Il appelle VehiculeModel::update.

Extrait de code (Orchestration - UserController.php)

```

<?php

// Extrait de UserController::updateVehicle()

public static function updateVehicle(PDO $pdo, int $userId, array $postData)
{
    // ... (Récupération des variables $marque, $modele, etc.)

    $vehicleId = $postData['id'] ?? null;
    $vehicleModel = new VehiculeModel($pdo);
    $success = false;

    if ($vehicleId) {

```

```

// --- LOGIQUE UPDATE ---
// 1. Le Contrôleur appelle le Modèle
$success = $vehiculeModel->update(
    (int)$vehiculeId,
    $userId, // L'ID de session est utilisé pour la sécurité
    $marque,
    $modele,
    // ... autres variables
    $date_immat
);
$message = 'Véhicule mis à jour.';
} else {
    // Logique du CREATE (vue ci-dessus)
    // ...
}

// ... (Réponse JSON)
}

```

Extrait de code (Accès aux données - VehiculeModel.php)

<?php

// Extrait de VehiculeModel::update()

```

public function update(int $vehiculeId, int $userId, string $marque, string
$modele, string $couleur, string $plaque, string $energie, string $immat):
bool
{
    // 2. Préparation de la requête UPDATE
    // Note : On ajoute "AND utilisateur_id = ?" à la clause WHERE.
    // C'est une sécurité cruciale pour empêcher un utilisateur
    // de modifier un véhicule qui ne lui appartient pas.
    $stmt = $this->pdo->prepare(
        "UPDATE vehicules SET marque = ?, modele = ?, couleur = ?, energie =
?,
        plaque_immatriculation = ?, date_premiere_immat = ?
        WHERE id = ? AND utilisateur_id = ?"
    );

    // 3. Exécution sécurisée
    return $stmt->execute([$marque, $modele, $couleur, $energie, $plaque,
$immat, $vehiculeId, $userId]);
}

```

4. DELETE (Logique) : Annuler un trajet

Dans cette application, une "suppression" est souvent une mise à jour de statut (une suppression logique) plutôt qu'une suppression physique (DELETE). L'annulation d'un trajet par un chauffeur est un parfait exemple.

- **Flux** : Le chauffeur clique sur "Annuler". L'API appelle `TrajetController::cancelTrajet`. Le contrôleur orchestre une transaction qui **met à jour** le statut du trajet, **lit** les passagers, et **rembourse** leurs crédits.

Extrait de code (Orchestration - TrajetController.php)

```
<?php

// Extrait de TrajetController::cancelTrajet()

public static function cancelTrajet(PDO $pdo, int $trajetId)
{
    // ... (Vérification de l'utilisateur)

    try {
        $pdo->beginTransaction();

        $trajetModel = new TrajetModel($pdo);
        $reservationModel = new ReservationModel($pdo);
        $userModel = new UserModel($pdo);

        // ... (Vérification des droits du chauffeur sur le trajet)

        // 1. Le Contrôleur appelle le Modèle pour faire l'UPDATE (le "Delete"
logique)
        $trajetModel->updateStatus($trajetId, 'annulé');

        // 2. Opérations liées : Remboursement des passagers
        $passagers = $reservationModel->getConfirmedPassagers($trajetId);

        foreach ($passagers as $passager) {
            $userModel->creditCredits($passager['utilisateur_id'],
$trajet['prix']);
            $reservationModel->updateStatus($passager['id'], 'annulée');
            // ... (Notifications)
        }

        $pdo->commit();
        // ... (Réponse JSON)
    } catch (\Exception $e) {
        $pdo->rollBack();
        // ... (Réponse JSON d'erreur)
    }
}
```

Extrait de code (Accès aux données - TrajetModel.php)

```
<?php
```

```
// Extrait de TrajetModel::updateStatus()
```

```
// Cette méthode simple est utilisée par le contrôleur
```

```
// pour effectuer la "suppression logique".
```

```
public function updateStatus(int $trajetId, string $newStatus): int
```

```
{
```

```
    $stmt = $this->pdo->prepare("UPDATE covoiturages SET statut = ? WHERE id =  
?");
```

```
    $stmt->execute([$newStatus, $trajetId]);
```

```
    return $stmt->rowCount();
```

```
}
```



7. Démonstration de la Transaction Métier

Démonstration d'une Transaction Métier (Validation de Trajet)

Pour illustrer l'architecture MVC et la gestion transactionnelle, le cas d'usage le plus pertinent est la **validation d'un trajet** par un passager (géré par `AvisController::validateTrajet`).

Cette action n'est pas une simple requête SQL, mais une **transaction métier** qui orchestre plusieurs modèles pour garantir l'intégrité des données.

Objectif Métier : Lorsqu'un passager valide un trajet, le système doit impérativement :

1. Mettre à jour le statut de sa réservation.
2. **Payer le chauffeur** en transférant les crédits.
3. Enregistrer l'avis du passager pour modération.

Sécurité (Transaction PDO) : Ces trois actions doivent réussir ou échouer *ensemble*. Il est impensable de payer le chauffeur si l'enregistrement de l'avis échoue. Pour garantir cela, l'ensemble du bloc est entouré d'une transaction PDO :

- `$pdo->beginTransaction()`; est appelé au début.
- Si tout réussit, `$pdo->commit()`; valide les changements.
- Si une seule erreur survient, un catch (`Exception $e`) exécute un `$pdo->rollBack()`; pour annuler toutes les opérations, assurant qu'aucun crédit n'est transféré à tort.

Orchestration (Le Contrôleur) : Le `AvisController` ne contient aucun SQL. Il agit en chef d'orchestre et appelle les méthodes de différents modèles pour accomplir la tâche :

Extrait simplifié de `AvisController::validateTrajet()` :

```
<?php
public static function validateTrajet(PDO $pdo, int $reservation_id, array
$avisData)
{
    // (Ici, on récupère $passagerId depuis $_SESSION)

    try {
        // 1. Démarrer la transaction
        $pdo->beginTransaction();

        // 2. Instancier les modèles
        $reservationModel = new ReservationModel($pdo);
        $avisModel = new AvisModel($pdo);
        $userModel = new UserModel($pdo);

        // 3. VÉRIFIER (Appel Modèle 1)
        // Le contrôleur vérifie si l'utilisateur a le droit
        $reservation = $reservationModel->findForValidation($reservation_id,
$passagerId);
        if (!$reservation) {
            throw new \Exception('Impossible de valider ce trajet.');
```

```

}

// 4. EXÉCUTER (Appels Modèles 2, 3 et 4)

// a. Mettre à jour la réservation
$reservationModel->updateStatus($reservation_id, 'validée');

// b. Insérer l'avis
$avisModel->create($passagerId, $reservation['covoiturage_id'],
(int)$avisData['note'], $avisData['commentaire']);

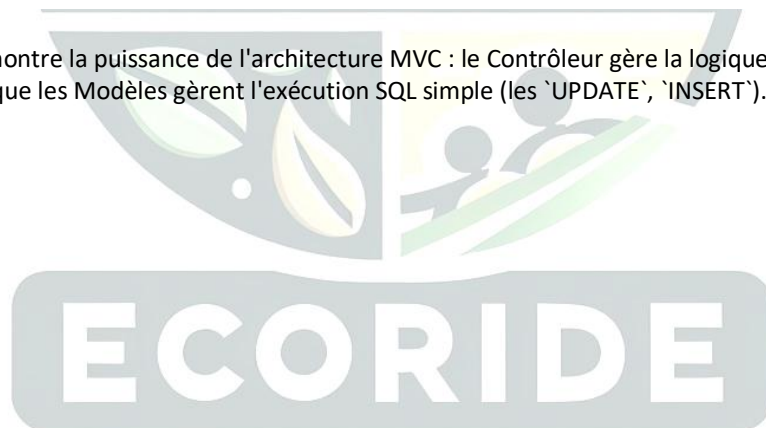
// c. Payer le chauffeur
$userModel->creditCredits($reservation['chauffeur_id'],
$reservation['prix']);

// 5. Valider la transaction
$pdo->commit();
echo json_encode(['success' => true, 'message' => 'Trajet validé !']);

} catch (\Exception $e) {
// 6. Annuler en cas d'erreur
$pdo->rollBack();
echo json_encode(['success' => false, 'message' => $e->getMessage()]);
}
}

```

Cette approche démontre la puissance de l'architecture MVC : le Contrôleur gère la logique métier complexe (la transaction) tandis que les Modèles gèrent l'exécution SQL simple (les `UPDATE`, `INSERT`).



8. Démonstration du Dynamisme Côté Client

Pour illustrer comment le frontend interagit avec le backend, la page du dashboard Administrateur utilise JavaScript natif (ES6+) pour charger les statistiques de manière asynchrone.

1. **L'Appel Asynchrone (fetch)** : Au chargement de la page, un script envoie une requête fetch à la route protégée /api/stats.
2. **La Réponse JSON** : Le AdminController (côté serveur) exécute les requêtes SQL (via AdminModel) et renvoie les données au format JSON.
3. **La Manipulation du DOM** : Le JavaScript, côté client, reçoit ce JSON et l'utilise pour instancier les objets new Chart() (de la librairie Chart.js) et remplir les graphiques.

Extrait de code (admin.js) :

```
// Ajoute un écouteur d'événement qui attend que le document HTML
// soit entièrement chargé et analysé par le navigateur avant d'exécuter le
// script.
document.addEventListener('DOMContentLoaded', function() {

    // 1. Appel Asynchrone de l'API backend
    // On utilise 'fetch' pour faire une requête HTTP GET vers la route
    // /api/stats.
    fetch('/api/stats')
        .then(response => {
            // 2. Première promesse : gérer la réponse HTTP
            // On vérifie si la réponse du serveur est un succès (ex: 200 OK).
            if (!response.ok) {
                // Si la réponse est une erreur (ex: 403 Forbidden, 404 Not Found,
                // 500 Server Error),
                // on lève une exception pour la transmettre au bloc .catch() plus
                // bas.
                throw new Error('Erreur réseau ou accès non autorisé');
            }
            // Si la réponse est OK, on la convertit du format JSON en un objet
            // JavaScript.
            // response.json() retourne lui-même une promesse.
            return response.json();
        })
        .then(data => {
            // 3. Deuxième promesse : gérer les données JSON
            // 'data' est maintenant un objet JavaScript contenant les
            // statistiques.

            // 3a. Préparation des données pour les graphiques
            // On extrait les labels (les jours) du premier jeu de données.
            const jours = data.covoiturages_par_jour.map(item => item.jour);
            // On extrait les données numériques pour le premier graphique.
            const nbCovoiturages = data.covoiturages_par_jour.map(item =>
            item.nombre_covoiturages);
            // On extrait les données numériques pour le second graphique.
```

```

        const creditsParJour = data.credits_par_jour.map(item =>
item.credits_gagnes);

        // 3b. Manipulation du DOM (création du 1er graphique)
        // On cible l'élément <canvas> avec l'ID 'graphCovoiturages'.
        // On instancie un nouvel objet Chart (de la librairie Chart.js).
        new Chart(document.getElementById('graphCovoiturages'), {
            type: 'bar', // Type de graphique
            data: {
                labels: jours, // Labels pour l'axe X
                datasets: [{
                    label: 'Covoiturages par jour',
                    data: nbCovoiturages, // Données pour l'axe Y
                    backgroundColor: 'rgba(75, 192, 192, 0.6)'
                }]
            }
        });

        // 3c. Manipulation du DOM (création du 2ème graphique)
        // On cible le second <canvas> avec l'ID 'graphCredits'.
        new Chart(document.getElementById('graphCredits'), {
            type: 'line', // Type de graphique
            data: {
                labels: jours, // Labels pour l'axe X
                datasets: [{
                    label: 'Crédits gagnés par jour',
                    data: creditsParJour, // Données pour l'axe Y
                    borderColor: 'rgba(255, 99, 132, 1)',
                    fill: false // Pour ne pas colorer sous la ligne
                }]
            }
        });

        // 3d. Remplissage du total
        // On met à jour le texte de l'élément 'totalCredits' avec la donnée
reçue.
        document.getElementById('totalCredits').textContent =
data.total_credits;
    })

    .catch(error => {
        // 4. Gestion des Erreurs
        // Ce bloc s'exécute si le 'fetch' échoue (ex: pas de connexion)
        // ou si on a levé une erreur manuellement (ex: !response.ok).
        console.error('Erreur de chargement des statistiques:', error);
        // Ici, on pourrait afficher un message d'erreur à l'administrateur
        // document.getElementById('dashboard-container').innerHTML = "Erreur
de chargement des stats.";
    });

```

```
});
```

Cette approche montre une application "réactive" où le PHP sert d'API et le JavaScript gère le rendu dynamique.



9. Diagrammes

Les trois diagrammes suivants modélisent l'architecture et le comportement de l'application EcoRide à différents niveaux, du concept fonctionnel à la structure du code :

1. **Le Diagramme de Cas d'Utilisation** : Il répond à la question "Qui fait quoi ?". Il présente la vue **fonctionnelle** du système, en identifiant les acteurs (Visiteur, Passager, Admin...) et les fonctionnalités qu'ils sont autorisés à utiliser (ex: Réserver un trajet, Modérer les avis) .
2. **Le Diagramme de Classes** : Il répond à la question "Comment le code est-il construit ?". Il illustre l'architecture **statique** de l'application, en montrant les classes PHP (UserController, TrajetModel, etc.), leurs méthodes principales, et leurs relations (architecture MVC, injection de dépendance) .
3. **Le Diagramme de Séquence** : Il répond à la question "Comment cela fonctionne-t-il en détail ?". Il montre le comportement **dynamique** d'un scénario spécifique (la réservation d'un trajet), en détaillant la chronologie des appels de méthodes entre les objets et la gestion des transactions (le try...catch avec commit ou rollBack) .

1. Diagramme de Cas d'Utilisation

Ce diagramme de cas d'utilisation présente une vue d'ensemble des fonctionnalités de la plateforme EcoRide et des interactions possibles pour chaque type d'acteur. Il identifie les différents acteurs du système et les actions (cas d'utilisation) qu'ils sont autorisés à effectuer, permettant de comprendre rapidement "qui fait quoi" au sein de l'application.

L'architecture des acteurs est basée sur l'**héritage** : Passager, Chauffeur, Employé et Admin sont toutes des spécialisations de l'acteur de base Utilisateur.

Description des Acteurs et de leurs Cas d'Utilisation

Acteur : Visiteur

- Description : Représente toute personne non authentifiée sur la plateforme.
- Actions principales :
 - S'inscrire ou Se connecter.
 - Rechercher et filtrer les trajets disponibles.
 - Consulter les détails d'un trajet et le profil public d'un chauffeur.

Acteur : Utilisateur

- Description : C'est l'acteur de base pour toute personne connectée. Les autres rôles (Passager, Chauffeur, etc.) héritent de ses actions.
- Actions principales :
 - Se déconnecter.
 - Gérer son profil (le modifier).
 - Acheter des crédits pour recharger son compte.

Acteur : Passager

- Description : Un Utilisateur ayant le rôle de passager. Il hérite de toutes les actions de l'Utilisateur.
- Actions principales :
 - Réserver un trajet (action qui inclut le sous-cas "Débiter crédits passager").
 - Annuler une de ses réservations (action qui inclut le sous-cas "Rembourser passager").
 - Valider un trajet terminé (action qui inclut les sous-cas "Laisser un avis" et "Payer le Chauffeur").

- Signaler un incident (action qui étend le cas "Valider un trajet").

 Acteur : Chauffeur

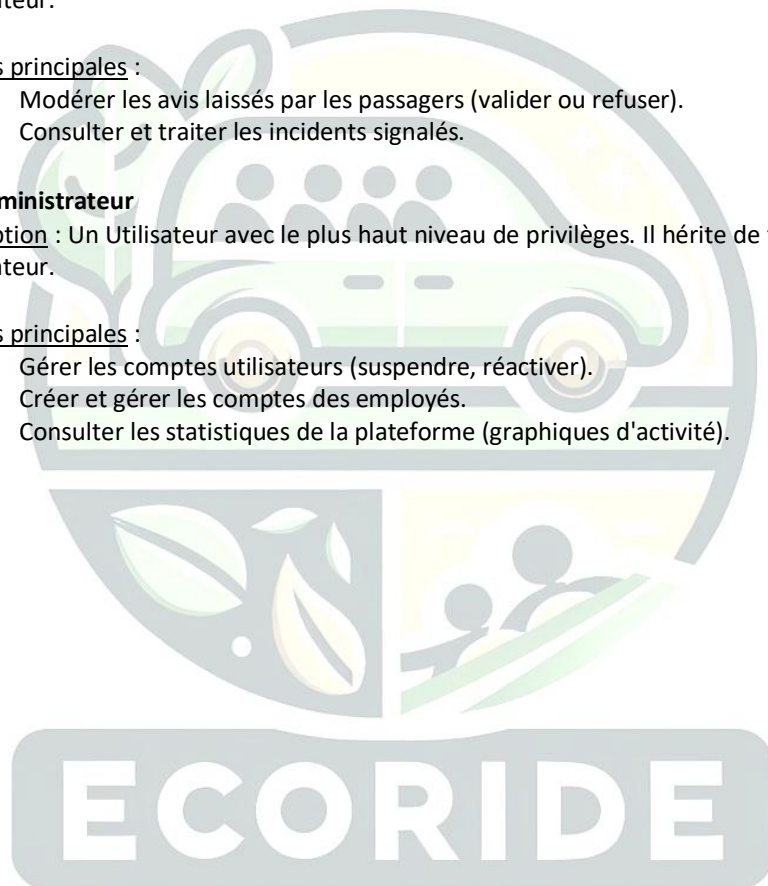
- Description : Un Utilisateur ayant le rôle de chauffeur. Il hérite de toutes les actions de l'Utilisateur.
- Actions principales :
 - Gérer ses véhicules (ajouter, modifier).
 - Proposer de nouveaux trajets (action qui inclut le sous-cas "Payer frais de publication").
 - Gérer les demandes de réservation (confirmer ou refuser).
 - Gérer le cycle de vie de ses trajets (Démarrer et Terminer).
 - Annuler un trajet qu'il a planifié (action qui inclut le sous-cas "Rembourser les passagers").

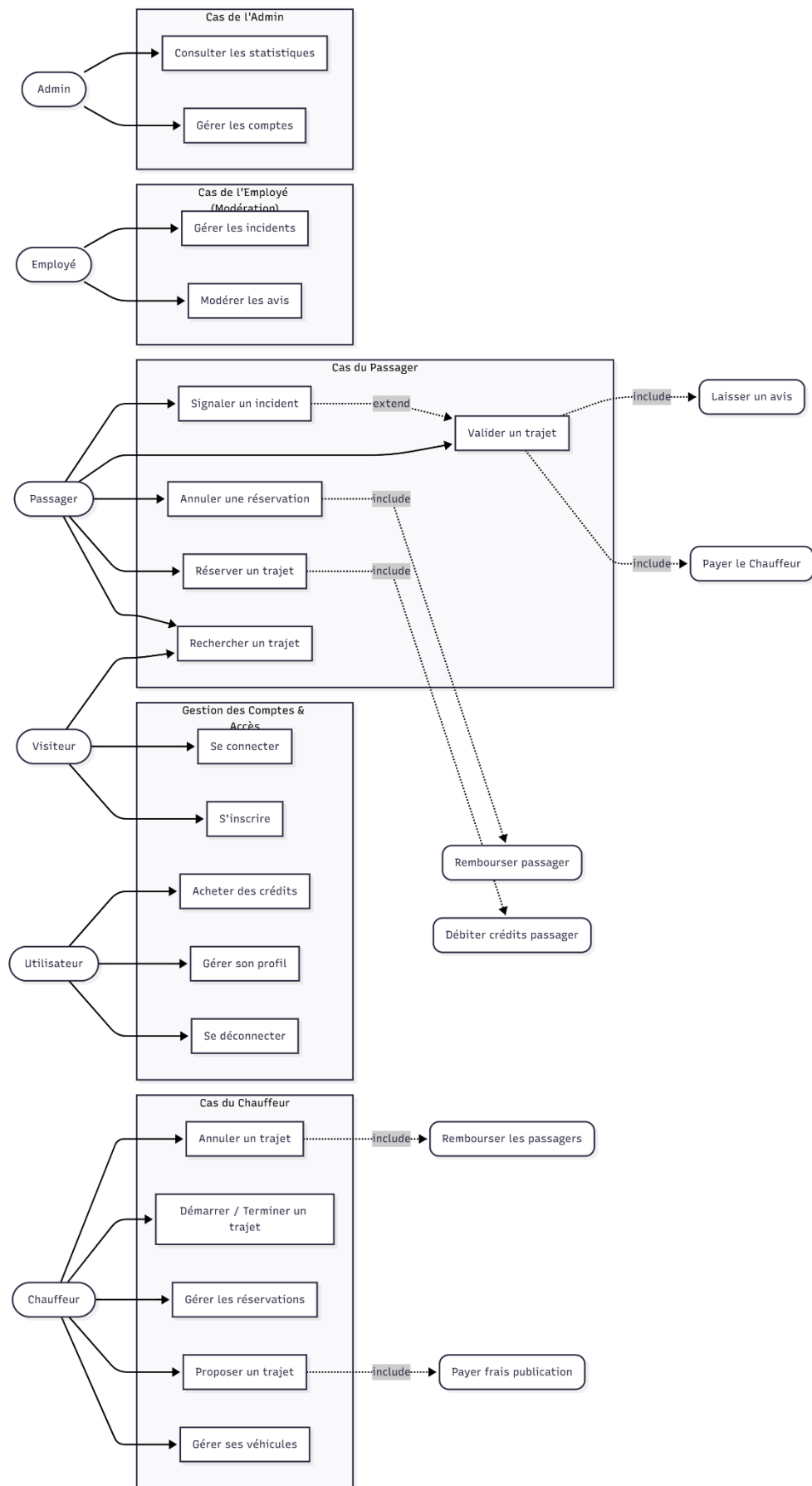
 Acteur : Employé

- Description : Un Utilisateur avec des droits de modération. Il hérite de toutes les actions de l'Utilisateur.
- Actions principales :
 - Modérer les avis laissés par les passagers (valider ou refuser).
 - Consulter et traiter les incidents signalés.

 Acteur : Administrateur

- Description : Un Utilisateur avec le plus haut niveau de privilèges. Il hérite de toutes les actions de l'Utilisateur.
- Actions principales :
 - Gérer les comptes utilisateurs (suspendre, réactiver).
 - Créer et gérer les comptes des employés.
 - Consulter les statistiques de la plateforme (graphiques d'activité).





2. Diagramme de classes

L'application est structurée selon une architecture MVC (Modèle-Vue-Contrôleur). Les classes principales sont réparties en trois couches distinctes : le Cœur (services), les Modèles (accès aux données) et les Contrôleurs (logique métier).

1. Cœur & Utilitaires

Ces classes fournissent les services de base à l'application.

Database : Classe utilitaire (Singleton) responsable de créer et de fournir l'unique instance de connexion à la base de données (PDO).

PDO : Classe intégrée de PHP. Elle est au centre de l'architecture car elle est injectée dans tous les modèles via leur constructeur.

2. Couche des Modèles (Models)

(Couche d'Accès aux Données)

Les Modèles sont la seule partie de l'application autorisée à communiquer avec la base de données. Chaque modèle reçoit une instance PDO et gère les requêtes SQL pour une entité spécifique.

UserModel : Gère la table utilisateurs. Responsable de l'authentification (findUserByEmail), de la gestion des crédits (getCreditsForUpdate, debitCredits), et de la création des utilisateurs.

ProfilModel : Gère la table profils_utilisateur. Corrige votre description : Chauffeur et Passager ne sont pas des classes. Ce modèle gère l'enregistrement des rôles (est_chauffeur, est_passager) liés à un UserModel.

CovoiturageModel : Gère la recherche et le filtrage des trajets (getFiltered, getNextAvailableDate).

TrajetModel : Gère la création et la gestion d'état des trajets (create, findByIdForUpdate, updateStatus, decrementPlaces).

VehiculeModel : Gère la table vehicules (création, mise à jour, et recherche par utilisateur).

ReservationModel : Gère la table reservations. Classe cruciale qui lie les utilisateurs aux trajets (create, updateStatus, findForValidation, getConfirmedPassagers).

AvisModel : Gère la table avis. Responsable de la création d'avis (create avec statut 'en_attente') et de leur récupération (getStatus, getChauffeurAvisValides).

IncidentModel : Gère la table incidents. Responsable de la création (create avec statut 'ouvert') et de la récupération (getStatus).

NotificationModel : Gère la table notifications (création, et récupération des non-lus).

3. Couche des Contrôleurs (Controllers)

(Couche de Logique Métier)

Les Contrôleurs orchestrent l'application. Ils gèrent les requêtes HTTP, valident les entrées, gèrent les transactions (beginTransaction, commit, rollback) et appellent les Modèles pour manipuler les données.

AuthController : Gère l'inscription et la connexion.

Exemple : La méthode `register()` utilise `UserModel` (pour créer l'utilisateur), `ProfilModel` (pour assigner les rôles) et `VehiculeModel` (si l'utilisateur est chauffeur).

TrajetController : Gère toutes les actions liées à un trajet.

Exemple : La méthode `participerTrajet()` gère une transaction complexe qui utilise `ReservationModel` (pour vérifier les doublons), `TrajetModel` (pour vérifier les places), `UserModel` (pour vérifier et déduire les crédits), puis `TrajetModel` à nouveau (pour décrémenter les places) et `ReservationModel` (pour créer la réservation).

AvisController : Gère le cycle de vie de la validation post-trajet.

Exemple : `validateTrajet()` utilise `ReservationModel` (pour trouver la réservation), `AvisModel` (pour créer l'avis) et `UserModel` (pour payer le chauffeur).

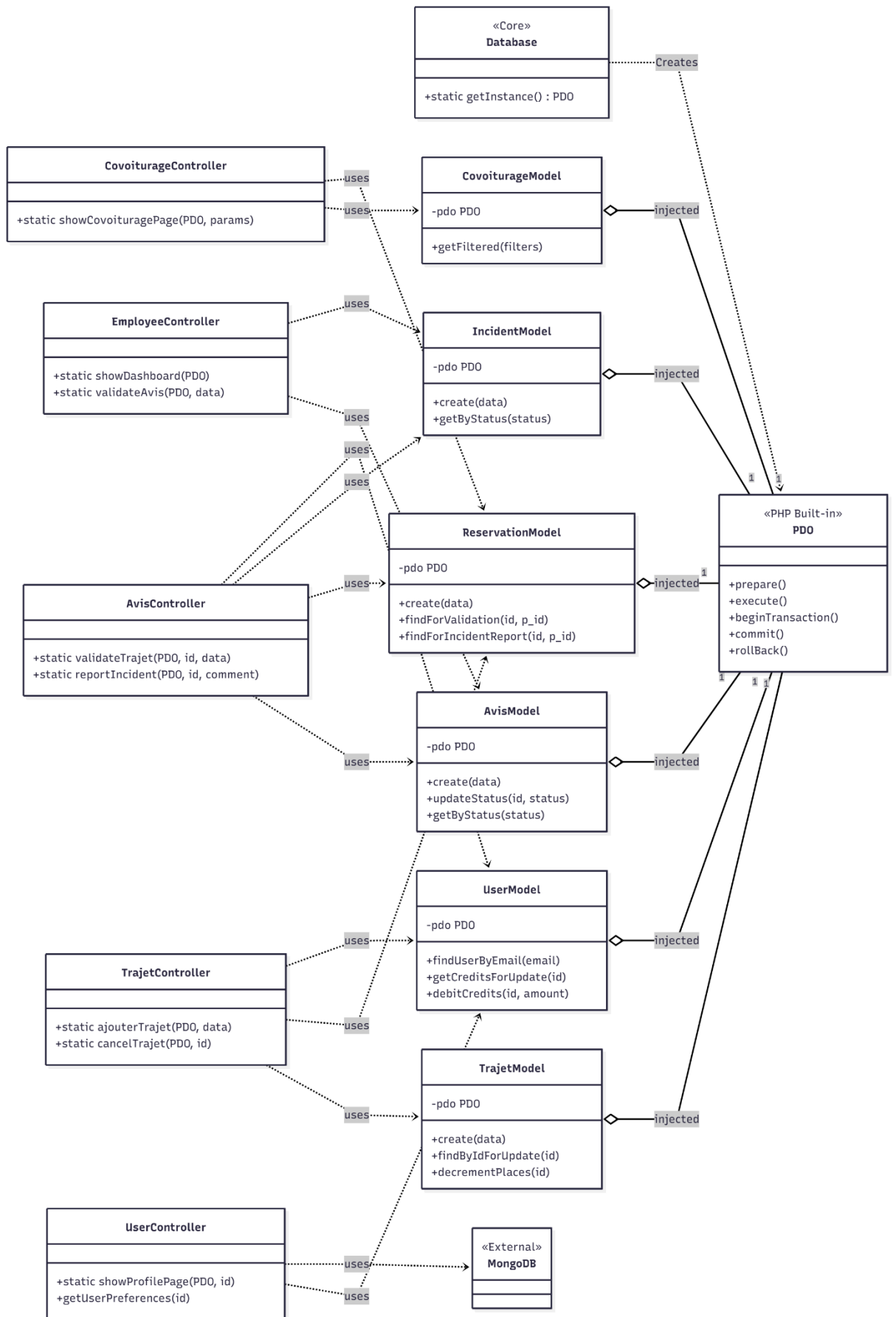
EmployeeController : Gère le tableau de bord des employés.

Exemple : `showDashboard()` utilise `AvisModel` et `IncidentModel` (en appelant `getByStatus()`) pour préparer les listes pour la vue.

UserController : Gère l'affichage et la modification du profil.

Exemple : `showProfilePage()` est un "chef d'orchestre" qui appelle de nombreux modèles (`UserModel`, `VehiculeModel`, `TrajetModel`, `ReservationModel`, `AvisModel`) pour agréger toutes les données nécessaires à la vue du profil.





3. Diagramme de Séquence

Ce diagramme de séquence illustre le scénario clé du cycle de vie d'un trajet sur la plateforme EcoRide. Il se décompose en deux transactions distinctes :

1. La finalisation du trajet par le chauffeur.
2. La validation du trajet par le passager (qui déclenche le paiement).

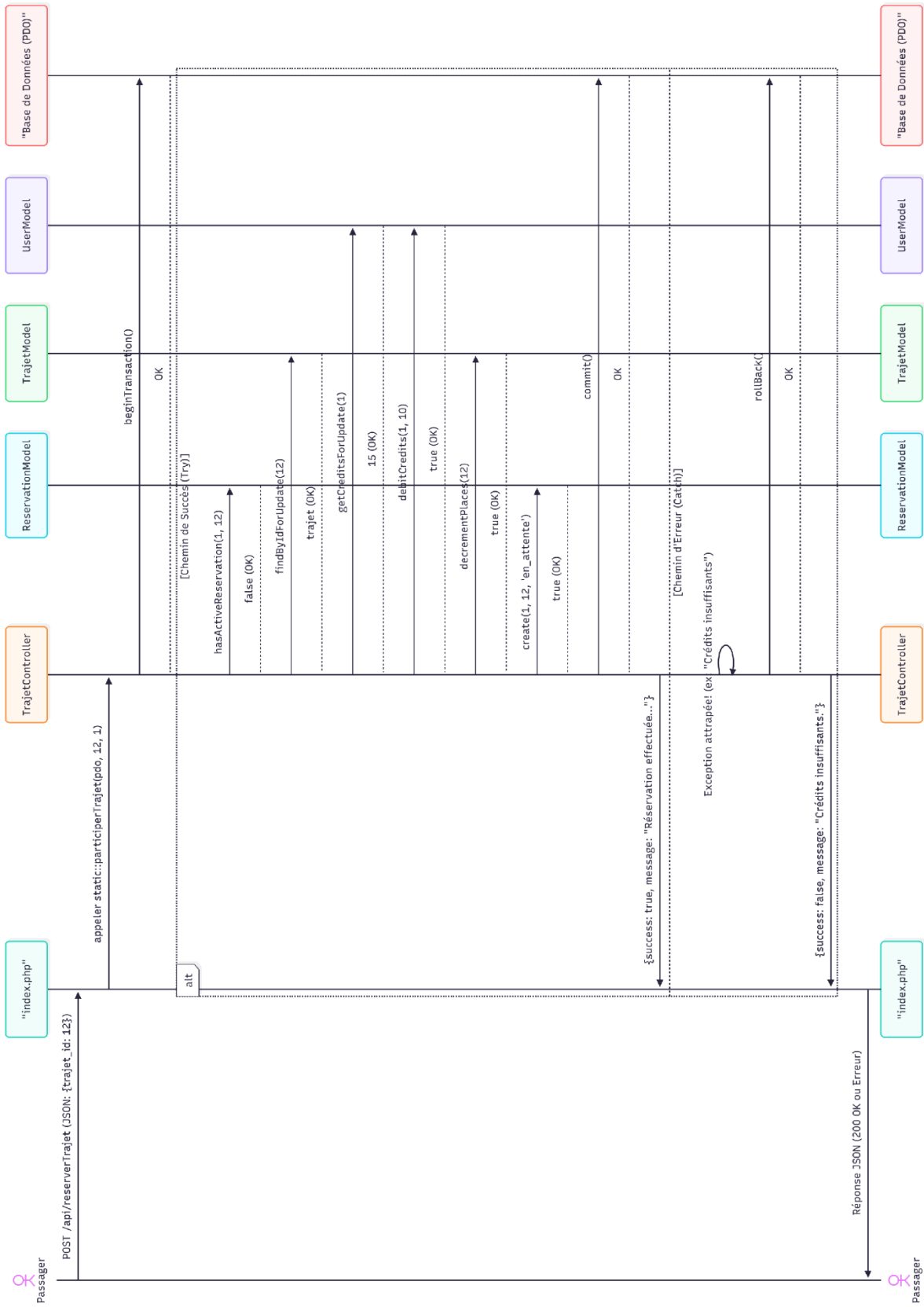
Partie 1 : Le Chauffeur termine le trajet

1. Le **Chauffeur** clique sur "Arrivée à destination" sur son interface (la Vue).
2. L'Interface envoie une requête fetch asynchrone à la route API /api/endTrajet.
3. Le **Routeur** (index.php) dirige l'appel vers `TrajetController::endTrajet`.
4. Le `TrajetController` démarre une **transaction PDO** pour garantir l'intégrité des données.
5. Il appelle `TrajetModel::updateStatusConditional()` pour changer le statut du covoiturage de "en_cours" à "terminé".
6. Il appelle `ReservationModel::getConfirmedPassagers()` pour récupérer la liste des passagers.
7. Il boucle sur les passagers et appelle `UserModel::createNotification()` (et le service d'email) pour inviter chaque **Passager** à valider le trajet.
8. Si tout réussit, la transaction est validée (commit) et une réponse de succès est renvoyée.

Partie 2 : Le Passager valide le trajet

9. Le **Passager**, notifié, se connecte à son profil et soumet le formulaire de notation (note + commentaire).
10. L'Interface envoie une requête fetch à la route /api/validateTrajet.
11. Le **Routeur** dirige l'appel vers `AvisController::validateTrajet`.
12. L'`AvisController` démarre une **transaction PDO** pour le paiement et la validation.
13. Il appelle `ReservationModel::findForValidation()` pour vérifier que le trajet est bien "terminé" et appartient au bon passager.
14. **(Actions de la transaction)** Le contrôleur exécute 3 actions critiques :
 - Il appelle `ReservationModel::updateStatus()` pour passer la réservation à "validée".
 - Il appelle `UserModel::creditCredits()` pour transférer les crédits du trajet au **Chauffeur**.
 - Il appelle `AvisModel::create()` pour enregistrer l'avis avec le statut "en_attente" de modération.
15. Si les 3 opérations réussissent, la transaction est validée (commit). Le système retourne un message de succès au **Passager**.

Le diagramme illustre également la **gestion des erreurs** (le bloc `alt...else`). Si une seule de ces opérations (par exemple, le paiement du chauffeur) échoue, une exception est levée, la transaction entière est annulée (`rollback`), et une réponse d'erreur est renvoyée à l'utilisateur.



10. Tests et Validation

Pour garantir la fiabilité de l'application, plusieurs niveaux de tests ont été effectués.

Tests d'Intégration (API)

Les points d'API critiques ont été testés manuellement à l'aide d'un outil comme Postman (ou l'inspecteur réseau du navigateur) pour valider :

- **La sécurité** : Tenter d'appeler `/api/stats` sans être connecté en tant qu'admin renvoie bien une erreur 403 (Forbidden).
- **La logique** : Appeler `/api/reserverTrajet` avec un ID de trajet invalide renvoie bien un JSON `{ "success": false, "message": "Trajet non trouvé." }`.

Tests Fonctionnels (Jeu d'essai utilisateur)

Un jeu d'essai a été suivi pour valider les scénarios métier principaux :

| Scénario | Acteur | Action | Résultat Attendu | Résultat Obtenu |
|-----------------------|------------------------------|--|--|-----------------|
| Réservation (Succès) | Passager (test@test.fr) | Réserve un trajet Paris-Lille avec 15 crédits. | L'utilisateur a 35 crédits. Le statut est "en_attente". | Conforme |
| Réservation (Échec) | Passager (passager@test.fr) | Tente de réserver un trajet à 25 crédits (il n'en a que 20). | L'API renvoie une erreur "Crédits insuffisants". La réservation n'est pas créée. | Conforme |
| Modération (Avis) | Employé (employe@ecoride.fr) | Valide un avis "en_attente" sur le trajet Lyon-Marseille. | L'avis devient "validé". Le TRIGGER SQL recalcule la note_moyenne du chauffeur (id 3). | Conforme |
| Modération (Incident) | Employé (employe@ecoride.fr) | Marque l'incident n°1 comme "fermé". | L'incident n°1 a le statut "fermé". | Conforme |
| Accès (Admin) | Visiteur (non connecté) | Tente d'accéder à /admin. | Est redirigé vers /login. | Conforme |

ECORIDE

11.Étapes de déploiement et gestion de version (Git)

Cette section décrit la démarche et les étapes suivies pour déployer l'application EcoRide sur un serveur de production, la rendant ainsi accessible en ligne.

○ Choix de la Plateforme d'Hébergement

Pour ce projet, la plateforme **Railway.app** a été choisie. Cette solution de type PaaS (Platform as a Service) a été sélectionnée pour plusieurs raisons :

- **Simplicité** : Elle permet de déployer une application directement depuis un dépôt GitHub avec une configuration minimale.
- **Offre Gratuite Complète** : Son plan gratuit est suffisant pour héberger les trois composants de notre stack : l'application PHP, la base de données MySQL, et la base de données MongoDB.
- **Gestion Centralisée** : Tous les services sont gérés au sein d'un seul et même projet, ce qui simplifie la configuration du réseau et des variables d'environnement.

○ Préparation du Projet pour le Déploiement

Avant de lancer le déploiement, plusieurs étapes de préparation du code ont été nécessaires pour assurer la compatibilité avec un environnement de production :

- **Configuration des dépendances (composer.json)** : Le fichier a été simplifié pour ne déclarer que les librairies PHP essentielles. Les dépendances d'extensions PHP (ext-*) ont été retirées pour laisser la plateforme de déploiement gérer l'installation de l'environnement PHP de base.
- **Gestion du Fichier composer.lock** : Le fichier composer.lock a été inclus dans le dépôt Git. C'est une étape cruciale qui garantit que le serveur de déploiement installe exactement les mêmes versions des dépendances que celles utilisées en développement, assurant ainsi la stabilité.
- **Préparation du fichier SQL (ecoride.sql)** : Le fichier d'exportation de la base de données a été modifié pour garantir une importation fiable :

Le fichier ecoride.sql a été **nettoyé** de tous les ajouts superflus de phpMyAdmin et **validé manuellement** pour garantir que la structure, les contraintes de clés étrangères et les TRIGGERS (pour les notes moyennes) s'exécutent proprement lors de l'importation.

Les commandes SET FOREIGN_KEY_CHECKS=0; et SET FOREIGN_KEY_CHECKS=1; ont été ajoutées au début et à la fin du fichier pour contourner les problèmes d'ordre lors de la création des tables.

- **Gestion des Variables d'Environnement (.env)** : Le fichier index.php a été modifié pour que le chargement du fichier .env soit conditionnel. Ainsi, en local, les variables du fichier sont utilisées, tandis qu'en production, l'application utilise les variables fournies par la plateforme.

○ Processus de Déploiement sur Railway.app

Le déploiement s'est déroulé en suivant ces étapes chronologiques :

- **Création du Projet et des Services** : Un projet vide a été créé sur Railway. Ensuite, les trois services nécessaires ont été ajoutés : une base de données MySQL, une base de données MongoDB, et l'application PHP déployée depuis le dépôt GitHub Fani1987/EcoRide.
- **Importation des Données** : Une connexion a été établie avec la base de données MySQL en ligne via un client SQL externe (HeidiSQL). Le fichier ecoride.sql a été exécuté pour créer la structure des tables et insérer les données de test. Cette étape a nécessité la configuration d'une connexion sécurisée (SSL).
- **Configuration des Variables d'Environnement** : C'est l'étape la plus critique. Dans l'onglet "Variables" du service applicatif sur Railway, toutes les variables d'environnement ont été configurées (DB_HOST, DB_USER, DB_PASS, DB_PORT, MONGO_..., MAIL_...) en utilisant les identifiants de connexion fournis par les services de base de données Railway.
- **Configuration de la Commande de Démarrage** : Dans les réglages du service applicatif, la "Start Command" a été définie sur `php -S 0.0.0.0:$PORT -t ..`. Cette commande lance le serveur web intégré de PHP et lui indique d'utiliser le port fourni par Railway, assurant que l'application est accessible depuis l'extérieur.

○ **Vérification Finale**

Après un déploiement réussi, un domaine public (ex: <https://ecoride-production-d9b8.up.railway.app>) a été généré via les réglages du service. L'application a ensuite été testée de fond en comble sur cette adresse en ligne pour valider le bon fonctionnement de toutes les fonctionnalités : inscription, connexion, recherche, réservation et les espaces spécifiques aux différents rôles.

Méthodologie de Gestion de Version (Git)

Le simple fait d'utiliser Git n'est pas suffisant ; il est crucial d'adopter un flux de travail (workflow) professionnel pour garantir la stabilité du projet.

Conformément aux bonnes pratiques, le projet n'a pas été développé sur une seule branche main, ce qui est risqué. J'ai appliqué un flux de travail basé sur les branches develop et main :

1. **Branche main** : Elle est protégée. Elle ne contient que le code de production stable. Le déploiement sur Railway était initialement lié à cette branche.
2. **Branche develop** : C'est la branche de travail principale. La plupart des 29 commits du projet s'y trouvent.
3. **Branches de feature / hotfix** : Pour toute nouvelle fonctionnalité ou correction (comme la mise en place du JavaScript de réservation), le travail est fait sur une branche dédiée (ex: hotfix/modal-reservations).

Exemple d'un ajout récent :

Pour l'ajout de la documentation finale du projet (les fichiers Annexes.pdf, manuel d'utilisation.pdf, ecoride.sql et les dernières corrections de code), le travail n'a pas été "pushé" directement sur develop.

Au lieu de cela :

1. Une branche dédiée, **feature/ajout-documentation-finale**, a été créée à partir de develop pour isoler le travail.

Bash

```
# On se place sur la branche de développement  
git checkout develop
```

```
# On crée la nouvelle branche de fonctionnalité et on bascule dessus  
git checkout -b feature/ajout-documentation-finale
```

2. Les fichiers ont été ajoutés et "commités" sur cette branche locale.

Bash

```
# On ajoute tous les nouveaux fichiers (PDFs, SQL, etc.)  
git add .
```

```
# On sauvegarde les changements avec un message clair  
git commit -m "Docs: Ajout de la documentation finale"
```

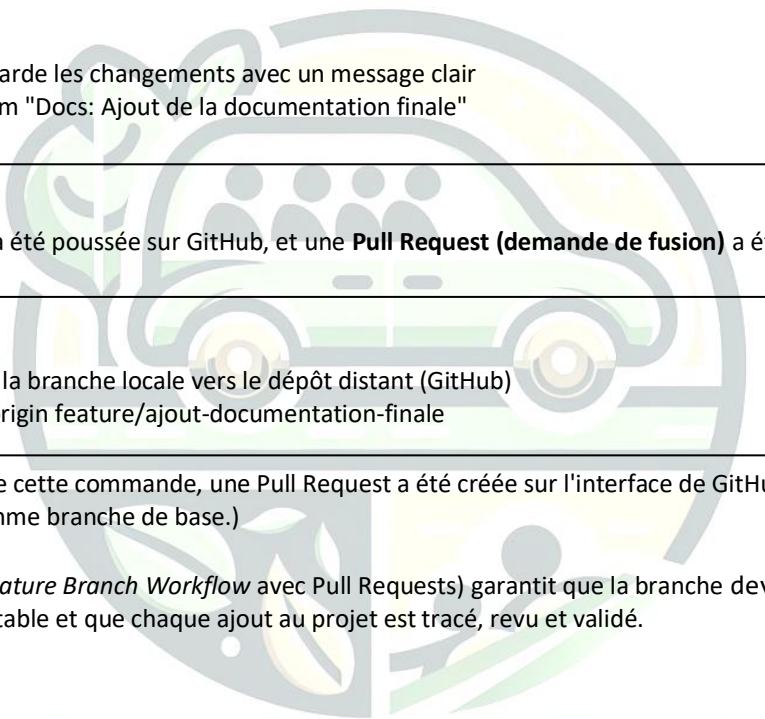
3. La branche a été poussée sur GitHub, et une **Pull Request (demande de fusion)** a été ouverte.

Bash

```
# On envoie la branche locale vers le dépôt distant (GitHub)  
git push -u origin feature/ajout-documentation-finale
```

(À la suite de cette commande, une Pull Request a été créée sur l'interface de GitHub, en ciblant develop comme branche de base.)

Cette méthode (le *Feature Branch Workflow* avec Pull Requests) garantit que la branche develop (et donc le déploiement) reste stable et que chaque ajout au projet est tracé, revu et validé.



ECORIDE